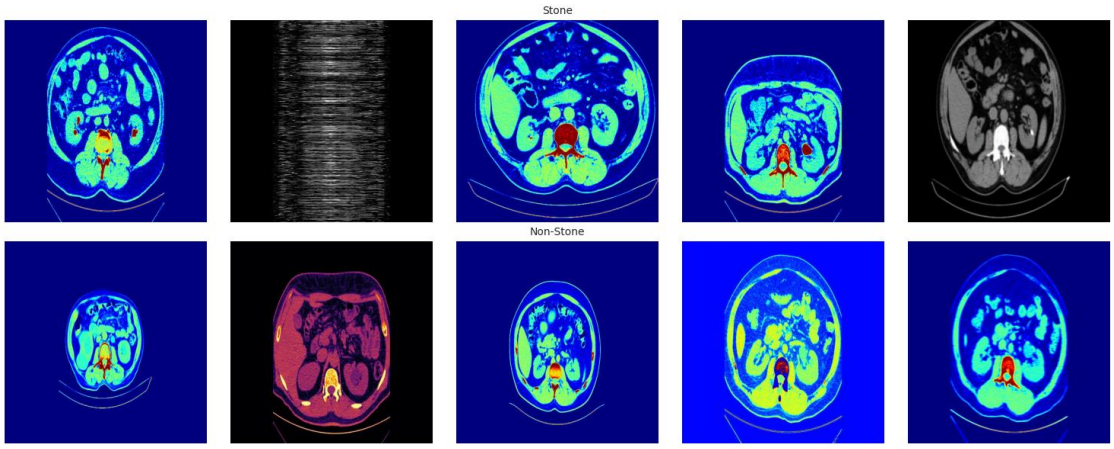
Probability-Aware Patch Selection

for Kidney Stone Detection (CT scans)




```
import numpy as np
import pandas as pd
import os

root_path = '/kaggle/input/kidney-stone-axial-ct-imaging-colorized-
mixeddata/Axial CT Imaging Dataset for AI-Powered Kidney Stone Detection A
Resource for Deep Learning Research dataset/train/'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df.head()
```

	image_path	label
0	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Stone
1	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Stone
2	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Stone
3	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Stone
4	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Stone

```
df.tail()
```

	image_path	label
1995	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Non-Stone
1996	/kaggle/input/kidney-stone-axial-ct-imaging-co...	Non-Stone

```
1997  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1998  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1999  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
```

```
df.shape
```

```
(2000, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
label         0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   image_path  2000 non-null   object
 1   label       2000 non-null   object
dtypes: object(2)
memory usage: 31.4+ KB
```

```
df['label'].unique()
```

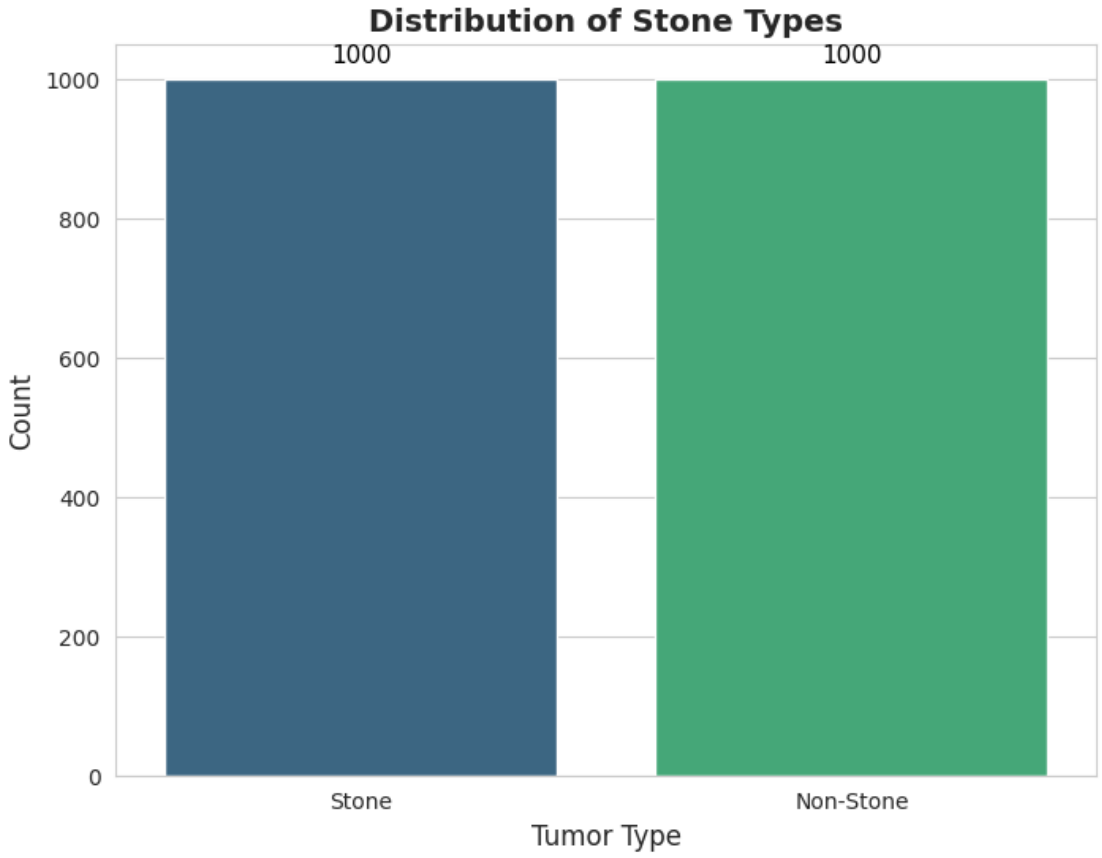
```
array(['Stone', 'Non-Stone'], dtype=object)
```

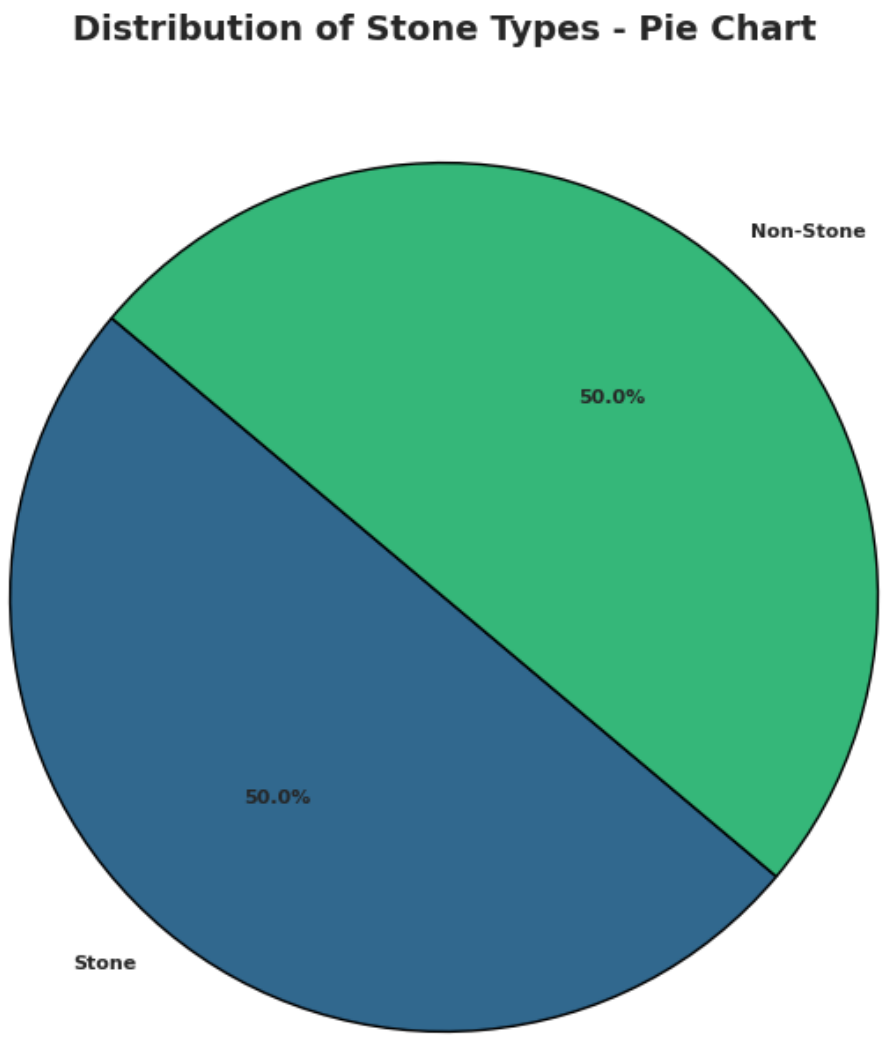
```
df['label'].value_counts()
```

```
label
Stone        1000
Non-Stone    1000
Name: count, dtype: int64
```

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Stone Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)
```

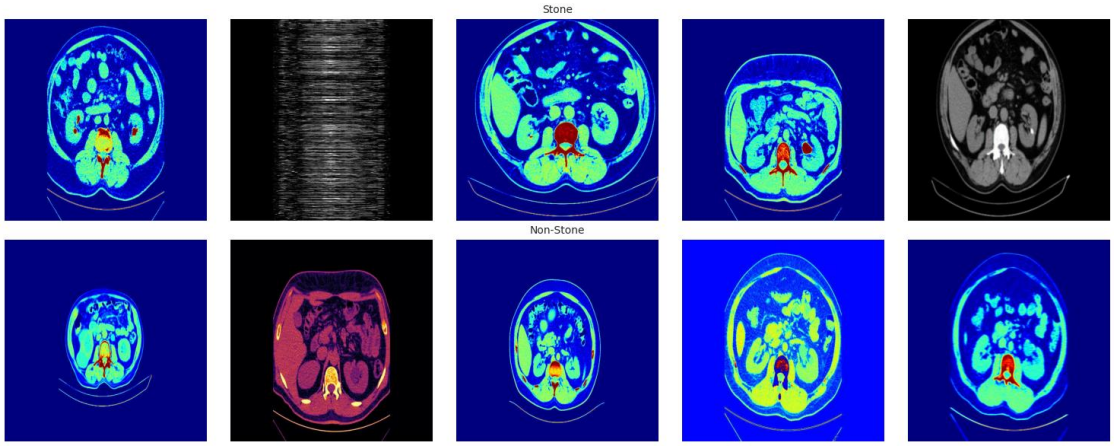
```
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
       startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
'bold'},
       wedgeprops={'edgecolor': 'black', 'linewidth': 1})

ax.set_title("Distribution of Stone Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()
```






```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
        plt.subplot(len(unique_labels), num_images, plt_idx)
        img = Image.open(img_path)
        plt.imshow(img)
```

```
        plt.axis('off')
        if col_idx == 2:
            plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()
```




```
df
```

```
                                                image_path       label
0     /kaggle/input/kidney-stone-axial-ct-imaging-co...       Stone
1     /kaggle/input/kidney-stone-axial-ct-imaging-co...       Stone
2     /kaggle/input/kidney-stone-axial-ct-imaging-co...       Stone
3     /kaggle/input/kidney-stone-axial-ct-imaging-co...       Stone
4     /kaggle/input/kidney-stone-axial-ct-imaging-co...       Stone
...                                                 ...         ...
1995  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1996  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1997  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1998  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone
1999  /kaggle/input/kidney-stone-axial-ct-imaging-co...  Non-Stone

[2000 rows x 2 columns]
```

ViT

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms, models
from torchvision.io import read_image
from torch.amp import GradScaler, autocast
from sklearn.metrics import confusion_matrix, classification_report,
roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import numpy as np
```

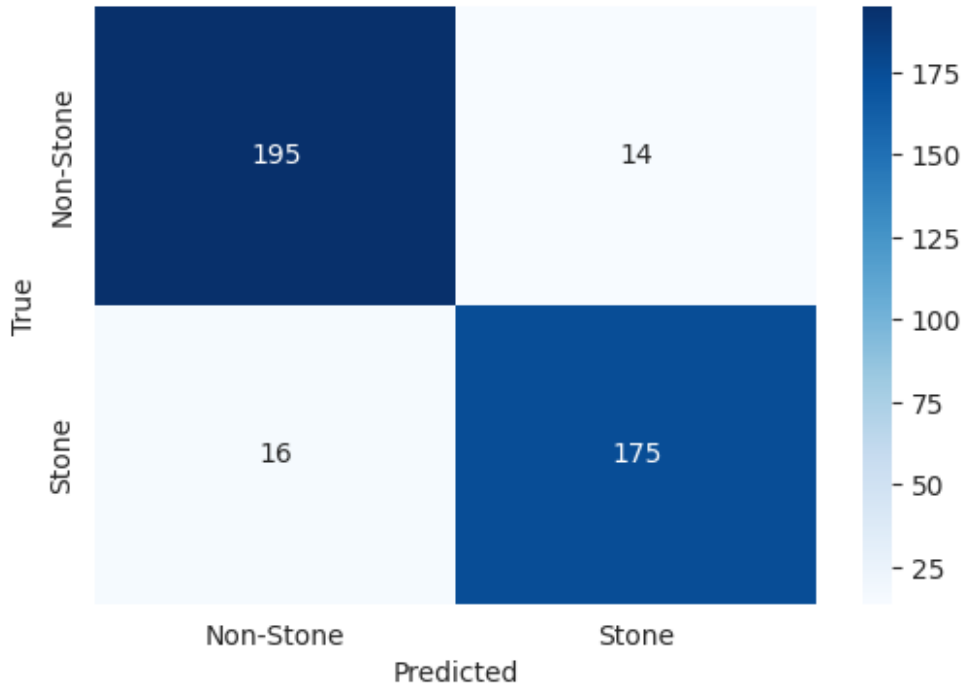
```
import pandas as pd

class KidneyStoneDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label = self.df.iloc[idx]['label_num']
        image = read_image(img_path)
        if image.shape[0] == 1:
            image = image.repeat(3, 1, 1)
        image = image.float() / 255.0
        if self.transform:
            image = self.transform(image)
        return image, label

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

df['label_num'] = df['label'].map({'Stone': 1, 'Non-Stone': 0})
full_dataset = KidneyStoneDataset(df)
train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size
train_data, test_data = random_split(full_dataset, [train_size, test_size])

train_data.dataset.transform = train_transform
test_data.dataset.transform = test_transform

train_loader = DataLoader(train_data, batch_size=32, shuffle=True,
num_workers=2, pin_memory=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False,
num_workers=2, pin_memory=True)

model = models.vit_b_16(weights='DEFAULT')
model.heads.head = nn.Linear(model.heads.head.in_features, 1)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-5, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)
scaler = GradScaler('cuda')

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in tqdm(loader):
        images, labels = images.to(device),
labels.to(device).float().unsqueeze(1)
        optimizer.zero_grad()
        with autocast('cuda'):
            outputs = model(images)
            loss = criterion(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        running_loss += loss.item()
        preds = (torch.sigmoid(outputs) > 0.5).float()
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return running_loss / len(loader), correct / total

def evaluate(model, loader, device):
    model.eval()
    preds, trues, probs = [], [], []
    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            prob = torch.sigmoid(outputs)
            probs.extend(prob.cpu().numpy())
            preds.extend((prob > 0.5).cpu().numpy())
            trues.extend(labels.numpy())
    return np.array(preds).flatten(), np.array(trues),
np.array(probs).flatten()

train_losses, train_accs, val_accs = [], [], []
for epoch in range(5):
    t_loss, t_acc = train_epoch(model, train_loader, criterion, optimizer,
device)
    v_preds, v_trues, v_probs = evaluate(model, test_loader, device)
    v_acc = (v_preds == v_trues).mean()
    train_losses.append(t_loss)
    train_accs.append(t_acc)
    val_accs.append(v_acc)
    scheduler.step()
    print(f"Epoch {epoch+1}: T_Loss:{t_loss:.4f} T_Acc:{t_acc:.4f}
V_Acc:{v_acc:.4f}")
```

```
test_preds, test_trues, test_probs = evaluate(model, test_loader, device)

cm = confusion_matrix(test_trues, test_preds)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Stone',
'Stone'], yticklabels=['Non-Stone', 'Stone'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

print(classification_report(test_trues, test_preds, target_names=['Non-
Stone', 'Stone']))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Loss')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(train_accs, label='Train Acc')
plt.plot(val_accs, label='Val Acc')
plt.legend()
plt.show()

fpr, tpr, _ = roc_curve(test_trues, test_probs)
plt.figure()
plt.plot(fpr, tpr, label=f'AUC = {auc(fpr, tpr):.2f}')
plt.plot([0, 1], [0, 1], '--')
plt.legend()
plt.show()

def get_attention_map(model, image, device):
    model.eval()
    img = image.unsqueeze(0).to(device)

    with torch.no_grad():
        x = model._process_input(img)
        n = x.shape[0]
        batch_class_token = model.class_token.expand(n, -1, -1)
        x = torch.cat([batch_class_token, x], dim=1)

        for block in model.encoder.layers:
            attn_output = block.self_attention(block.ln_1(x), block.ln_1(x),
block.ln_1(x))[0]
            x = x + block.dropout(attn_output)
            x = x + block.mlp(block.ln_2(x))

        final_attn = torch.mean(x[:, 1:, :], dim=-1).squeeze().cpu().numpy()
        normalized_attn = (final_attn - final_attn.min()) / (final_attn.max()
- final_attn.min())
    return normalized_attn

def patient_strategy(p_i, t_wait=1.0, t_to_D=None):
```

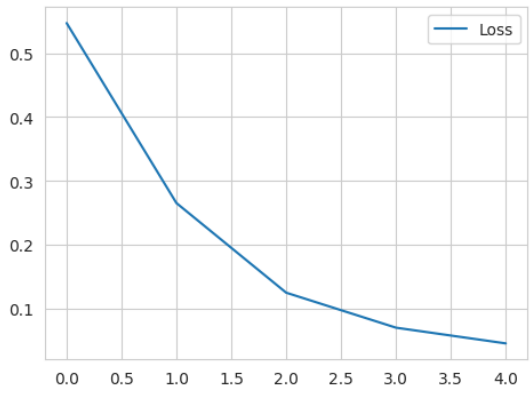
```
    if t_to_D is None:
        t_to_D = np.arange(len(p_i))
    V = -t_to_D - (1 / (p_i + 1e-6)) * t_wait
    i_star = np.argmax(V)
    return i_star, V[i_star]

demo_img, _ = test_data[0]
p_i = get_attention_map(model, demo_img, device)
i_star, V_star = patient_strategy(p_i)
print(f"Optimal patch: {i_star}, Value: {V_star:.2f}")
```

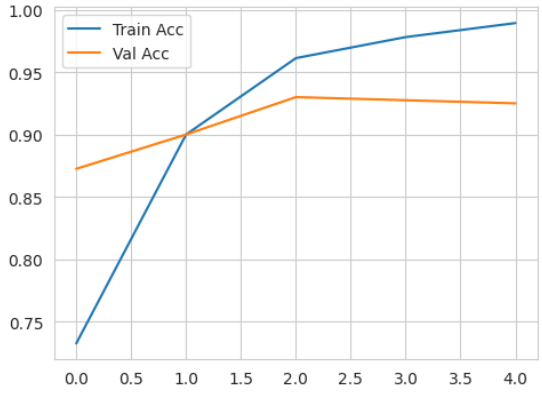
```
100%|██████████| 50/50 [00:17<00:00,  2.78it/s]

Epoch 1: T_Loss:0.5472 T_Acc:0.7325 V_Acc:0.8725

100%|██████████| 50/50 [00:19<00:00,  2.58it/s]

Epoch 2: T_Loss:0.2654 T_Acc:0.9000 V_Acc:0.9000

100%|██████████| 50/50 [00:17<00:00,  2.79it/s]

Epoch 3: T_Loss:0.1250 T_Acc:0.9613 V_Acc:0.9300

100%|██████████| 50/50 [00:17<00:00,  2.82it/s]

Epoch 4: T_Loss:0.0702 T_Acc:0.9781 V_Acc:0.9275

100%|██████████| 50/50 [00:18<00:00,  2.76it/s]

Epoch 5: T_Loss:0.0456 T_Acc:0.9894 V_Acc:0.9250
```

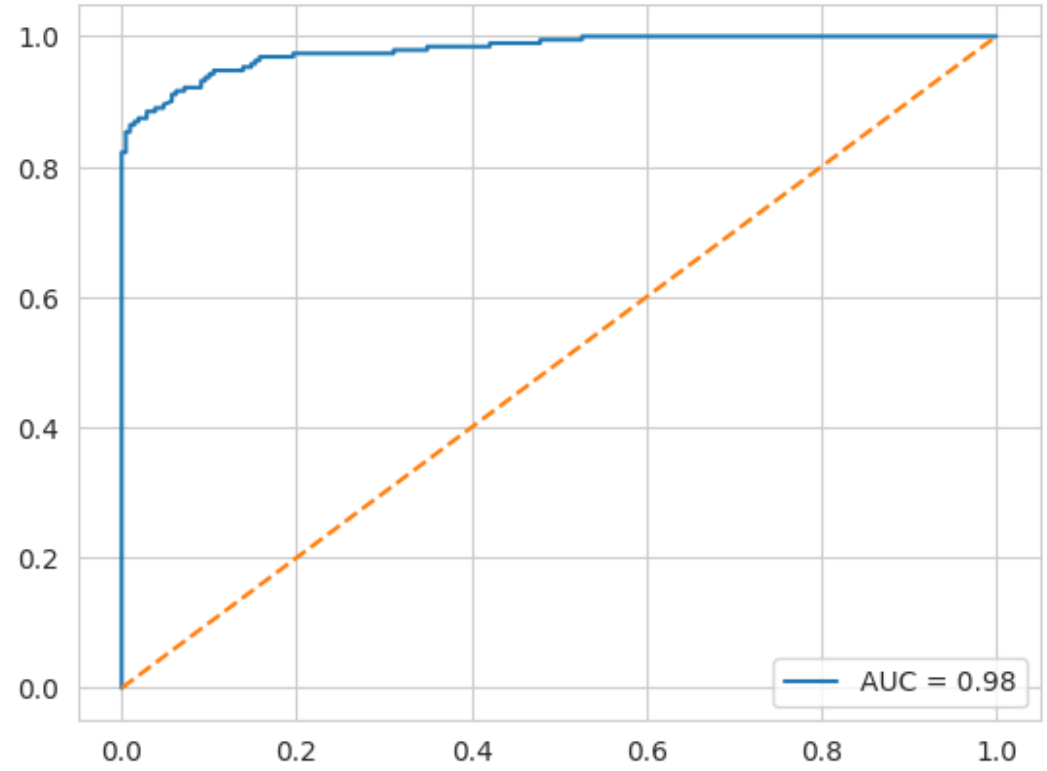



```
              precision    recall  f1-score   support

   Non-Stone       0.92      0.93      0.93       209
```

```
       Stone       0.93      0.92      0.92       191

    accuracy                           0.93       400
   macro avg       0.93      0.92      0.92       400
weighted avg       0.93      0.93      0.92       400
```



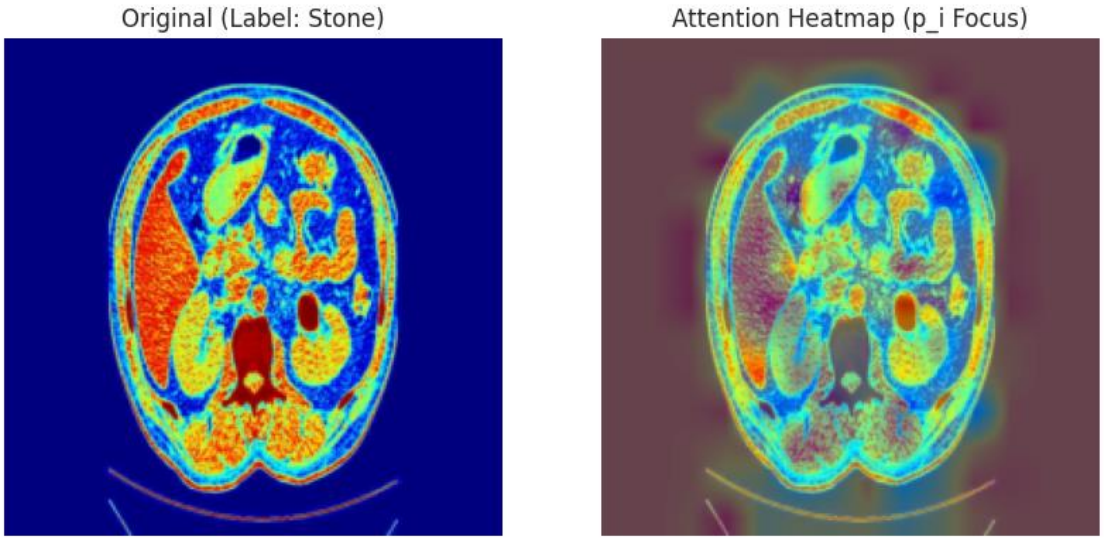





```
Optimal patch: 0, Value: -3.47
```

```python
import cv2

def plot_attention_overlay(model, dataset, idx, device):
    img_tensor, label = dataset[idx]

    p_i = get_attention_map(model, img_tensor, device)

    attn_grid = p_i.reshape(14, 14)
```

```
    heatmap = cv2.resize(attn_grid, (224, 224))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    orig_img = img_tensor.permute(1, 2, 0).cpu().numpy()
    orig_img = (orig_img * std + mean) * 255
    orig_img = np.uint8(np.clip(orig_img, 0, 255))

    overlay = cv2.addWeighted(orig_img, 0.6, heatmap, 0.4, 0)

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(orig_img)
    plt.title(f"Original (Label: {'Stone' if label==1 else 'Non-Stone'})")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(overlay)
    plt.title("Attention Heatmap (p_i Focus)")
    plt.axis('off')
    plt.show()

plot_attention_overlay(model, test_data, 0, device)
```



Swin ViT

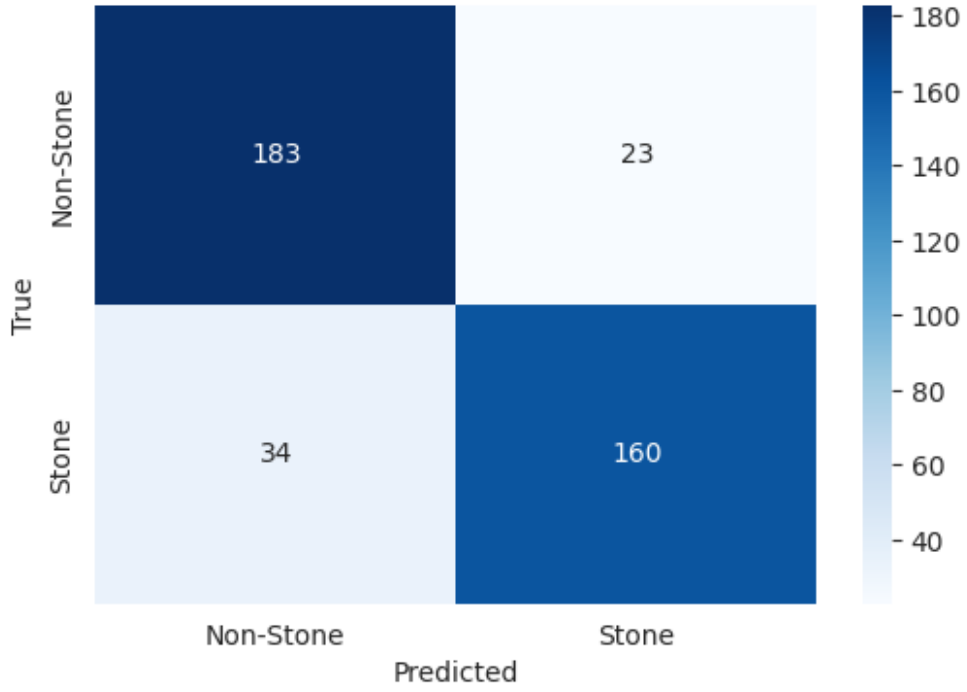
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms, models
from torchvision.io import read_image
```

```
from torch.amp import GradScaler, autocast
from sklearn.metrics import confusion_matrix, classification_report,
roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

class KidneyStoneDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label = self.df.iloc[idx]['label_num']
        image = read_image(img_path)
        if image.shape[0] == 1:
            image = image.repeat(3, 1, 1)
        image = image.float() / 255.0
        if self.transform:
            image = self.transform(image)
        return image, label

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(15),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

df['label_num'] = df['label'].map({'Stone': 1, 'Non-Stone': 0})
full_dataset = KidneyStoneDataset(df)
train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size
train_data, test_data = random_split(full_dataset, [train_size, test_size])

train_data.dataset.transform = train_transform
test_data.dataset.transform = test_transform

train_loader = DataLoader(train_data, batch_size=32, shuffle=True,
num_workers=2, pin_memory=True)
```

```
test_loader = DataLoader(test_data, batch_size=32, shuffle=False,
num_workers=2, pin_memory=True)

model = models.swin_t(weights='DEFAULT')
model.head = nn.Linear(model.head.in_features, 1)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-5, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)
scaler = GradScaler('cuda')

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in tqdm(loader):
        images, labels = images.to(device),
labels.to(device).float().unsqueeze(1)
        optimizer.zero_grad()
        with autocast('cuda'):
            outputs = model(images)
            loss = criterion(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        running_loss += loss.item()
        preds = (torch.sigmoid(outputs) > 0.5).float()
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return running_loss / len(loader), correct / total

def evaluate(model, loader, device):
    model.eval()
    preds, trues, probs = [], [], []
    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            prob = torch.sigmoid(outputs)
            probs.extend(prob.cpu().numpy())
            preds.extend((prob > 0.5).cpu().numpy())
            trues.extend(labels.numpy())
    return np.array(preds).flatten(), np.array(trues),
np.array(probs).flatten()

train_losses, train_accs, val_accs = [], [], []
for epoch in range(5):
    t_loss, t_acc = train_epoch(model, train_loader, criterion, optimizer,
device)
    v_preds, v_trues, v_probs = evaluate(model, test_loader, device)
    v_acc = (v_preds == v_trues).mean()
    train_losses.append(t_loss)
```

```
    train_accs.append(t_acc)
    val_accs.append(v_acc)
    scheduler.step()
    print(f"Epoch {epoch+1}: T_Loss:{t_loss:.4f} T_Acc:{t_acc:.4f}
V_Acc:{v_acc:.4f}")

test_preds, test_trues, test_probs = evaluate(model, test_loader, device)

cm = confusion_matrix(test_trues, test_preds)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Stone',
'Stone'], yticklabels=['Non-Stone', 'Stone'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

print(classification_report(test_trues, test_preds, target_names=['Non-
Stone', 'Stone']))

def get_swin_attention(model, image, device):
    model.eval()
    img = image.unsqueeze(0).to(device)
    with torch.no_grad():
        x = model.features(img)
        x = model.norm(x)
        x = model.permute(x)
        attn_map = torch.mean(x, dim=1).squeeze().cpu().numpy()
        normalized_attn = (attn_map - attn_map.min()) / (attn_map.max() -
attn_map.min() + 1e-8)
        return normalized_attn.flatten()

def patient_strategy(p_i, t_wait=1.0):
    t_to_D = np.arange(len(p_i))
    V = -t_to_D - (1 / (p_i + 1e-6)) * t_wait
    i_star = np.argmax(V)
    return i_star, V[i_star]

demo_img, _ = test_data[0]
p_i = get_swin_attention(model, demo_img, device)
i_star, V_star = patient_strategy(p_i)
print(f"Optimal patch (Swin 7x7 Grid): {i_star}, Value: {V_star:.2f}")

Downloading: "https://download.pytorch.org/models/swin_t-704ceda3.pth" to
/root/.cache/torch/hub/checkpoints/swin_t-704ceda3.pth
100%|██████████| 108M/108M [00:00<00:00, 191MB/s]
100%|██████████| 50/50 [00:12<00:00,  4.15it/s]

Epoch 1: T_Loss:0.6621 T_Acc:0.6075 V_Acc:0.6550

100%|██████████| 50/50 [00:11<00:00,  4.20it/s]

Epoch 2: T_Loss:0.5788 T_Acc:0.7094 V_Acc:0.7675

100%|██████████| 50/50 [00:12<00:00,  4.13it/s]
```

```
Epoch 3: T_Loss:0.4906 T_Acc:0.7869 V_Acc:0.8150

100%|██████████| 50/50 [00:11<00:00,  4.27it/s]

Epoch 4: T_Loss:0.4275 T_Acc:0.8131 V_Acc:0.8525

100%|██████████| 50/50 [00:11<00:00,  4.34it/s]

Epoch 5: T_Loss:0.3999 T_Acc:0.8319 V_Acc:0.8575
```



	precision	recall	f1-score	support
Non-Stone	0.84	0.89	0.87	206
Stone	0.87	0.82	0.85	194
accuracy			0.86	400
macro avg	0.86	0.86	0.86	400
weighted avg	0.86	0.86	0.86	400

```
Optimal patch (Swin 7x7 Grid): 0, Value: -1.02
```

Max ViT

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms, models
from torchvision.io import read_image
from torch.amp import GradScaler, autocast
from sklearn.metrics import confusion_matrix, classification_report,
roc_curve, auc
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import numpy as np
import pandas as pd

class KidneyStoneDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label = self.df.iloc[idx]['label_num']
        image = read_image(img_path)
        if image.shape[0] == 1:
            image = image.repeat(3, 1, 1)
        image = image.float() / 255.0
        if self.transform:
            image = self.transform(image)
        return image, label

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(20),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

df['label_num'] = df['label'].map({'Stone': 1, 'Non-Stone': 0})
full_dataset = KidneyStoneDataset(df)
train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size
train_data, test_data = random_split(full_dataset, [train_size, test_size])

train_data.dataset.transform = train_transform
test_data.dataset.transform = test_transform

train_loader = DataLoader(train_data, batch_size=32, shuffle=True,
num_workers=2, pin_memory=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False,
```

```
num_workers=2, pin_memory=True)

model = models.maxvit_t(weights='DEFAULT')
in_features = model.classifier[5].in_features
model.classifier[5] = nn.Linear(in_features, 1)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.AdamW(model.parameters(), lr=5e-5, weight_decay=0.05)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)
scaler = GradScaler('cuda')

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in tqdm(loader):
        images, labels = images.to(device),
labels.to(device).float().unsqueeze(1)
        optimizer.zero_grad()
        with autocast('cuda'):
            outputs = model(images)
            loss = criterion(outputs, labels)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        running_loss += loss.item()
        preds = (torch.sigmoid(outputs) > 0.5).float()
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return running_loss / len(loader), correct / total

def evaluate(model, loader, device):
    model.eval()
    preds, trues, probs = [], [], []
    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images)
            prob = torch.sigmoid(outputs)
            probs.extend(prob.cpu().numpy())
            preds.extend((prob > 0.5).cpu().numpy())
            trues.extend(labels.numpy())
    return np.array(preds).flatten(), np.array(trues),
np.array(probs).flatten()

for epoch in range(5):
    t_loss, t_acc = train_epoch(model, train_loader, criterion, optimizer,
device)
    v_preds, v_trues, v_probs = evaluate(model, test_loader, device)
    v_acc = (v_preds == v_trues).mean()
    scheduler.step()
    print(f"Epoch {epoch+1}: T_Loss:{t_loss:.4f} T_Acc:{t_acc:.4f}
```

```
V_Acc:{v_acc:.4f}")

test_preds, test_trues, test_probs = evaluate(model, test_loader, device)
print(classification_report(test_trues, test_preds, target_names=['Non-
Stone', 'Stone']))

def get_maxvit_attention(model, image, device):
    model.eval()
    img = image.unsqueeze(0).to(device)
    with torch.no_grad():
        x = model.stem(img)
        for block in model.blocks:
            x = block(x)
        attn_map = torch.mean(x, dim=1).squeeze().cpu().numpy()
        normalized_attn = (attn_map - attn_map.min()) / (attn_map.max() -
attn_map.min() + 1e-8)
        return normalized_attn.flatten()

demo_img, _ = test_data[0]
p_i = get_maxvit_attention(model, demo_img, device)
print(f"MaxViT Features Extracted. Feature Map Size: {len(p_i)}")

def patient_strategy(p_i, t_wait=1.0):
    t_to_D = np.arange(len(p_i))
    V = -t_to_D - (1 / (p_i + 1e-6)) * t_wait
    i_star = np.argmax(V)
    return i_star, V[i_star]

i_star, V_star = patient_strategy(p_i)
print(f"Optimal patch: {i_star}, Value: {V_star:.2f}")

100%|██████████| 50/50 [00:20<00:00,  2.41it/s]

Epoch 1: T_Loss:0.6388 T_Acc:0.6194 V_Acc:0.7575

100%|██████████| 50/50 [00:21<00:00,  2.34it/s]

Epoch 2: T_Loss:0.4718 T_Acc:0.8063 V_Acc:0.8525

100%|██████████| 50/50 [00:20<00:00,  2.45it/s]

Epoch 3: T_Loss:0.3346 T_Acc:0.8575 V_Acc:0.8950

100%|██████████| 50/50 [00:20<00:00,  2.46it/s]

Epoch 4: T_Loss:0.2392 T_Acc:0.9056 V_Acc:0.8925

100%|██████████| 50/50 [00:20<00:00,  2.42it/s]

Epoch 5: T_Loss:0.2022 T_Acc:0.9263 V_Acc:0.8950
              precision    recall  f1-score   support

   Non-Stone       0.88      0.91      0.89       194
       Stone       0.91      0.88      0.90       206
```

```
    accuracy                           0.90       400
   macro avg       0.90      0.90      0.89       400
weighted avg       0.90      0.90      0.90       400

MaxViT Features Extracted. Feature Map Size: 49
Optimal patch: 0, Value: -1.00
```